



# Mouse, the Language



[URL: http://primepuzzle.com/mouse/mouse7.html](http://primepuzzle.com/mouse/mouse7.html)

Last changed on Monday, April 23, 2007

When you test Mouse programs, occasionally things don't work at the beginning. Let's face it; things never work at the beginning!

If something goes wrong during the run, a usually fairly unreadable display of your code that has the problem is shown on the screen with a little caret just under the offending command. The nature of the error (like "Calculation stack underflowed") is also shown.

Usually, these errors just make you stare at your code and make you think harder about what might be wrong. You then make a change and run it again.

Mouse has a primitive trace feature which you can turn on by putting a left curly brace ( { ) in your code where you want tracing to start during a test run and a right curly brace ( } ) where you want it to stop. Quoting from page 73 of "MOUSE: A LANGUAGE FOR microcomputers"

"If *tracing* is *true*, the interpreter displays selected state information between the time when a character is fetched from the program by *getchar* and the time when it is processed by the CASE statement. In tracing mode the interpreter displays 80 characters of the Mouse program of which the fortieth character is the character about to be processed."

Here's code from the Mouse interpreter (the interpreter that comes with this tutorial was written in Pascal) that may clarify the above (and hopefully does not confuse!)

```
PROCEDURE getchar;
  BEGIN
    IF charpos < proglen
      THEN
        BEGIN
          charpos:=charpos+1;
          ch:=prog[charpos]
        END
      ELSE error(1)
    END; {getchar}
```

```
PROCEDURE interpret;
  BEGIN
    charpos:=0;
    tracing:=false;
    REPEAT
      getchar;
```

```

    IF tracing AND (ch<>' ')
      THEN display(charpos);
    CASE ch OF
      { Action for each symbol }
    END {CASE}
  UNTIL (ch='$')
END; {interpret}

```

Here's a demonstration of the trace feature.

---

```

~ trace.mse
{
  "Here's a very short program to show you what the trace feature will do.!!"
  "!Next line.!!"
  "!Please press your Enter key ...!!" ?'
}
"!I'm outta here."
$

```

---

```

B0>mouse-p
MOUSE, Peter Grogono, 1983; Lee Bradley, 2007
Enter Mouse program's name : trace.mse
                                { "Here's a very short program to show you
                                ^
Here's a very short program to show you what the trace feature will do.
ou what the trace feature will do.!!"  "!Next line.!!"  "!Please press your Ent
                                ^
Next line.

  feature will do.!!"  "!Next line.!!"  "!Please press your Enter key ...!!" ?'
                                ^
Please press your Enter key ...

  "!Please press your Enter key ...!!" ?'  }  "!I'm outta here."  $
                                ^
Please press your Enter key ...!!" ?'  }  "!I'm outta here."  $
                                ^

I'm outta here.
B0>

```

I hope you have enjoyed this tutorial. If you got this far, you're probably the kind of person that might want to try your own hand at writing something in Mouse. Please send

**me a note if you find an error in these pages or have something you'd like to share!**

**Lee Bradley, 24 East Cedar Street, Newington, CT 06111-2534**

**website : <http://primepuzzle.com/>**

- 0 - Emulator, Interpreter and the classic Hello, World! program
- 1 - Output string, output character, new line, Mouse's Stack, push character, end program
- 2 - Loops, variables, assignment, evaluation, comparison, comments, arithmetic, input number, output number
- 3 - Introduction to defining and calling macros
- 4 - If, input character, more on macros
- 5 - Recursive macros, the remainder operator, the small, whole number limitation of Mouse
- 6 - A few remarks about CP/M and MYZ80
- 7 - Testing and the trace feature, contact information